LAB 1:

CF Foundations

การกรองร่วมบน MovieMini-12

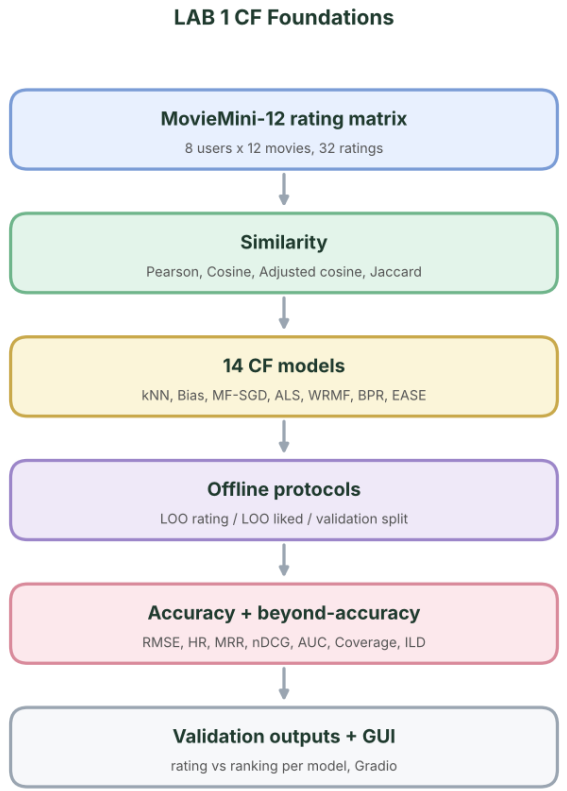
204729

Recommender Systems

ผู้ช่วยศาสตราจารย์ ดร.ดุษฎี ประเสริฐธิติพงษ์

Rating Matrix - 14 Models

Offline Evaluation - GUI



อธิบายแนวคิดของ Source Code ทีละ Part

Similarity - kNN - MF - EASE - LOO - Metrics - Gradio

COMPUTER SCIENCE

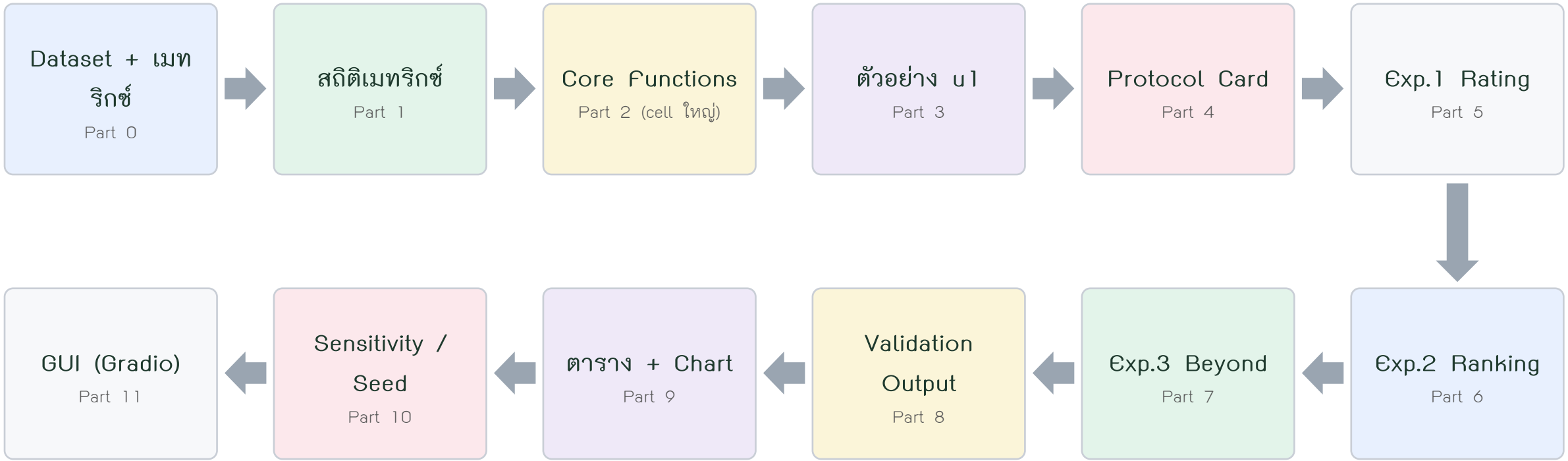
LAB 1 คืออะไร

ภาพรวม

- Notebook ตามบทที่ 5: สร้างระบบแนะนำแบบ **การกรองร่วม (Collaborative Filtering)** จากเมทริกซ์คะแนนอย่างเดียว ไม่ใช้เนื้อหาของภาพยนตร์
- ข้อมูล **MovieMini-12** ชุดเดียวกับบทที่ 3–4: ผู้ใช้ 8 คน ภาพยนตร์ 12 เรื่อง คะแนน 32 ค่า (อ่านจาก Drive หรือใช้ชุดที่ฝังใน code)
- สร้าง **14 โมเดล** ที่มี interface เดียวกัน: Baseline 4 ตัว, Bias, User/Item-kNN, MF-SGD, ALS, WRMF, BPR, EASE
- ออกแบบการทดลองแบบ Offline 3 ชุด: **Rating** (LOO 32 ครั้ง), **Ranking** (LOO liked 20 กรณี + ค่าคาดหมายเมื่อคะแนนเสมอ), **Beyond-Accuracy**
- แสดง **ตัวอย่าง Output บน Validation** เทียบทุกโมเดลทั้งค่าทำนายคะแนนและ Top-3 พร้อมตาราง/กราฟสรุป
- ปิดท้ายด้วย **GUI (Gradio)**: ข้อมูลผู้ใช้ ประวัติ รายการแนะนำ เหตุผล (เพื่อนบ้าน) และตารางเทียบทุกโมเดล

ไฟล์: 01_Lab_CF_MovieMini12_Models_Offline_Evaluation_GUI.ipynb

โครงสร้าง Notebook LAB 1 (Part 0 → Part 11)



Part 2 นิยามทุกฟังก์ชัน (ความคล้าย • โมเดล • metric) ที่ Part 3–11 เรียกใช้

แผนที่ของ Source Code LAB 1

Part	หน้าที่	ฟังก์ชัน / ตัวแปรสำคัญ	ผลลัพธ์
0	โหลดข้อมูล + เมทริกซ์ R	`load_dataset`, `build_matrix`, `R_FULL`, `G_MAT`	R ขนาด 8×12 (NaN = ว่าง)
1	สถิติพื้นฐาน	`known`, `co_rated`	density 0.3333, heatmap
2	Core Functions	`pearson_users`, `Recommender`, 13 คลาส, `tie_aware_metrics`	เครื่องมือทั้งหมด
3	ตัวอย่าง u1	`fitted_full`, `table`	ค่าทำนายทุกโมเดล
4	Protocol + Split	`RATING_FOLDS`, `RANK_FOLDS`, `validation_split`	32 / 20 folds, VAL 8 ค่า
5–7	การทดลอง 1–3	`evaluate_rating`, `evaluate_ranking`	rating / ranking / beyond
8	Output บน Validation	`fitted_val`, `val_rating`, `rank_view`	ตารางเทียบทุกโมเดล
9–10	สรุป + การทดลองเสริม	`summary`, `sens`, `paired_bootstrap`	chart, CI
11	GUI	`gui_recommend`, `gr.Blocks`	แอปทดลองใช้งาน

Part 0: Environment และแหล่งข้อมูล

```
!pip -q install gradio

import math, random, warnings, itertools, time
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display

warnings.filterwarnings("ignore")
pd.set_option("display.width", 200)
pd.set_option("display.max_columns", 40)
SEED = 2569                       # เมล็ดสุ่มเดียวกับในบทที่ 5

# ------------------------------------------------------------
# 0.1 พยายามอ่าน Dataset จาก Google Drive (ชุดเดียวกับ LAB บทที่ 3-4)
#     ถ้าไม่พบจะใช้ MovieMini-12 ที่ฝังไว้ใน code (ตารางที่ 5.3)
# ------------------------------------------------------------
DATA_DIR = None
try:
    from google.colab import drive
    drive.mount("/content/drive")
    BASE = Path("/content/drive/MyDrive/Colab Notebooks")
    DATA_ROOT = BASE / "Dataset_Chapter3_4"
    DATA_DIR = DATA_ROOT / "data" if (DATA_ROOT / "data").exists() else
        DATA_ROOT
except Exception as e:
    print("ไม่ได้รันบน Colab หรือ mount Drive ไม่ได้ →
ใช้ข้อมูลที่ฝังไว้", type(e).__name__)
```

อธิบายโค้ด

เตรียม environment + เลือกแหล่งข้อมูล

- ติดตั้ง **gradio** สำหรับ GUI ส่วน numpy / pandas / matplotlib มีใน Colab แล้ว
- **SEED = 2569** เมล็ดสุ่มเดียวกับบทที่ 5 ⟶ ผล MF/BPR ทำซ้ำได้
- พยายาม mount Drive และใช้ **Dataset_Chapter3_4** แบบเดียวกับ LAB บทที่ 3–4
- ถ้าไม่ได้รันบน Colab หรือไม่มีโฟลเดอร์ ⟶ **DATA_DIR = None** แล้วใช้ข้อมูลที่ฝังไว้แทน (notebook รันได้ทุกที่)

Part 0: MovieMini-12 ที่ฝังไว้ (ตารางที่ 5.3)

```
BUILTIN_ITEMS = pd.DataFrame({
    "item_id": [f"i{k}" for k in range(1, 13)],
    "title": ["Toy Story 5", "Inside Out 2", "The Wild Robot",
        "It Ends with Us", "Wicked",
              "Bridget Jones: Mad About the Boy", "Ballerina",
                  "Furiosa: A Mad Max Saga",
              "Conclave", "Dune: Part Two", "Mickey 17",
                  "Project Hail Mary"],
    "genre": ["Animation|Family", "Animation|Family",
        "Animation|Sci-Fi", "Drama|Romance",
              "Fantasy|Musical", "Drama|Romance",
                  "Action|Thriller", "Action|Sci-Fi",
              "Drama|Thriller", "Action|Sci-Fi",
                  "Sci-Fi|Thriller", "Drama|Sci-Fi"]})
BUILTIN_RATINGS = pd.DataFrame(
    [("u1","i1",5),("u1","i2",5),("u1","i3",4),("u1","i4",1),("u
        1","i6",1),
     ("u2","i1",2),("u2","i4",5),("u2","i5",4),("u2","i6",5),
     ("u3","i7",1),("u3","i10",5),("u3","i11",5),("u3","i12",4),
     ("u4","i1",4),("u4","i5",1),("u4","i7",5),("u4","i8",5),
     ("u5","i3",4),("u5","i6",3),("u5","i9",5),("u5","i10",2),
     ("u6","i2",5),("u6","i5",4),("u6","i11",3),
     ("u7","i3",3),("u7","i8",2),("u7","i11",4),("u7","i12",5),
     ("u8","i2",2),("u8","i4",4),("u8","i6",5),("u8","i11",3)],
    columns=["user_id","item_id","rating"])
```

อธิบายโค้ด

ข้อมูลชุดเดียวกับตารางที่ 5.3

- **BUILTIN_ITEMS**: 12 เรื่อง i1 Toy Story 5 … i12 Project Hail Mary พร้อมแนวภาพยนตร์
- **BUILTIN_RATINGS**: 32 แถว (user, item, rating) — ข้อมูลเข้าเพียงอย่างเดียวของการกรองร่วม
- แนวภาพยนตร์ **ไม่ได้ใช้แนะนำ** ใช้เพียงวัด ILD และ Calibration
- ตรวจได้ทันที: u1 ชอบ i1–i3 (แอนิเมชัน) ไม่ชอบ i4, i6

Part 0: load dataset + ปรับ schema

```python
def load_dataset():
    if DATA_DIR is not None and (DATA_DIR / "metadata.csv").exists()
        and (DATA_DIR / "ratings.csv").exists():
        items = pd.read_csv(DATA_DIR / "metadata.csv")
        ratings = pd.read_csv(DATA_DIR / "ratings.csv")
        for old, new in
            {"movie_id":"item_id","id":"item_id","movie_title":"title","
            name":"title",
                        "genres":"genre"}.items():
            if old in items.columns and new not in items.columns:
                items = items.rename(columns={old:new})
        for old, new in
            {"uid":"user_id","user":"user_id","iid":"item_id","movie_id"
            :"item_id",
                        "score":"rating"}.items():
            if old in ratings.columns and new not in ratings.columns:
                ratings = ratings.rename(columns={old:new})
        print("อ่านข้อมูลจาก", DATA_DIR)
        return items, ratings, "Drive"
    print("ใช้ MovieMini-12 ที่ฝังไว้ (ตารางที่ 5.3)")
    return BUILTIN_ITEMS.copy(), BUILTIN_RATINGS.copy(), "Built-in"

items, ratings, DATA_SOURCE = load_dataset()
items["item_id"] = items["item_id"].astype(str)
items["genre"] = items["genre"].fillna("").astype(str).str.replace(",",
    "|")
ratings["user_id"] = ratings["user_id"].astype(str)
ratings["item_id"] = ratings["item_id"].astype(str)
ratings["rating"] = pd.to_numeric(ratings["rating"]).astype(float)
```

อธิบายโค้ด

อ่านข้อมูลแบบทนทานต่อชื่อคอลัมน์

- rename เฉพาะเมื่อยังไม่มีชื่อมาตรฐาน (**item_id**, **title**, **genre**, **user_id**, **rating**)
- คืนค่า **DATA_SOURCE** = "Drive" หรือ "Built-in" เพื่อพิมพ์ใน log
- แปลงรหัสเป็น **str** ทุกตาราง ⟶ join กันได้ไม่ผิดชนิด
- แนวคั่นด้วย **|** เสมอ (แปลง **,** เป็น **|**)

Part 0: ลำดับรหัส + เมทริกซ์ R

```python
def natural_key(s):
    digits = "".join(ch for ch in str(s) if ch.isdigit())
    return (int(digits) if digits else 0, str(s))

USERS = sorted(ratings["user_id"].unique(), key=natural_key)
ITEMS = sorted(items["item_id"].unique(), key=natural_key)
items = items.set_index("item_id").loc[ITEMS].reset_index()
U_INDEX = {u:k for k, u in enumerate(USERS)}
I_INDEX = {i:k for k, i in enumerate(ITEMS)}
TITLE = dict(zip(items["item_id"], items["title"]))
GENRE = dict(zip(items["item_id"], items["genre"]))

# ------------------------------------------------------------
# 0.2 เมทริกซ์ผู้ใช้-รายการ R: ช่องว่าง = NaN (ไม่รู้) ไม่ใช่ศูนย์
# ------------------------------------------------------------
def build_matrix(df):
    R = np.full((len(USERS), len(ITEMS)), np.nan)
    for u, i, r in
        df[["user_id","item_id","rating"]].itertuples(index=False):
        R[U_INDEX[u], I_INDEX[i]] = r
    return R

R_FULL = build_matrix(ratings)
LIKE_T = 4.0                    # กติกาเดียวกับบทที่ 3-4: คะแนน 4-5
    = "ชอบ"
```

อธิบายโค้ด

เมทริกซ์ผู้ใช้–รายการ R

- `natural_key` เรียง i2 ก่อน i10 ⟶ คอลัมน์ตรงกับตารางในหนังสือ
- `U_INDEX`, `I_INDEX` แปลงรหัสเป็นแถว/คอลัมน์
- ช่องว่างเป็น NaN = ไม่รู้ ไม่ใช่ 0 (หัวใจของ MF ที่คิดเฉพาะช่องที่รู้ค่า)
- `LIKE_T = 4.0`: คะแนน 4–5 = ชอบ ใช้กับโมเดลข้อมูลโดยปริยายและ ranking

Part 0: เวกเตอร์แนว G_MAT สำหรับวัดผล

```
ALL_GENRES = sorted({g for s in items["genre"] for g in
    s.split("|") if g})
G_MAT = np.array([[1.0 if g in GENRE[i].split("|") else
    0.0 for g in ALL_GENRES] for i in ITEMS])

print(f"Source={DATA_SOURCE}  Users={len(USERS)}
    Items={len(ITEMS)}
    Ratings={int((~np.isnan(R_FULL)).sum())}")
display(pd.DataFrame(R_FULL, index=USERS,
    columns=ITEMS).fillna("·"))
display(items)
```

อธิบายโค้ด

แนวภาพยนตร์ใช้วัด ไม่ใช้แนะนำ

- G_MAT ขนาด 12×9: multi-hot ของแนวภาพยนตร์
- ใช้ใน ild() (ความต่างของแนวในรายการ) และ calibration_genre()
- พิมพ์จำนวนผู้ใช้/รายการ/คะแนน แล้วแสดงเมทริกซ์ (· = ว่าง) ให้ตรวจกับตารางที่ 5.3

Part 1: สถิติพื้นฐานของเมทริกซ์ (5.2.2)

```
known = ~np.isnan(R_FULL)
co_rated = np.array([[int((known[a] & known[b]).sum()) for b
    in range(len(USERS))] for a in range(len(USERS))])
pairs = [co_rated[a, b] for a, b in
    itertools.combinations(range(len(USERS)), 2)]

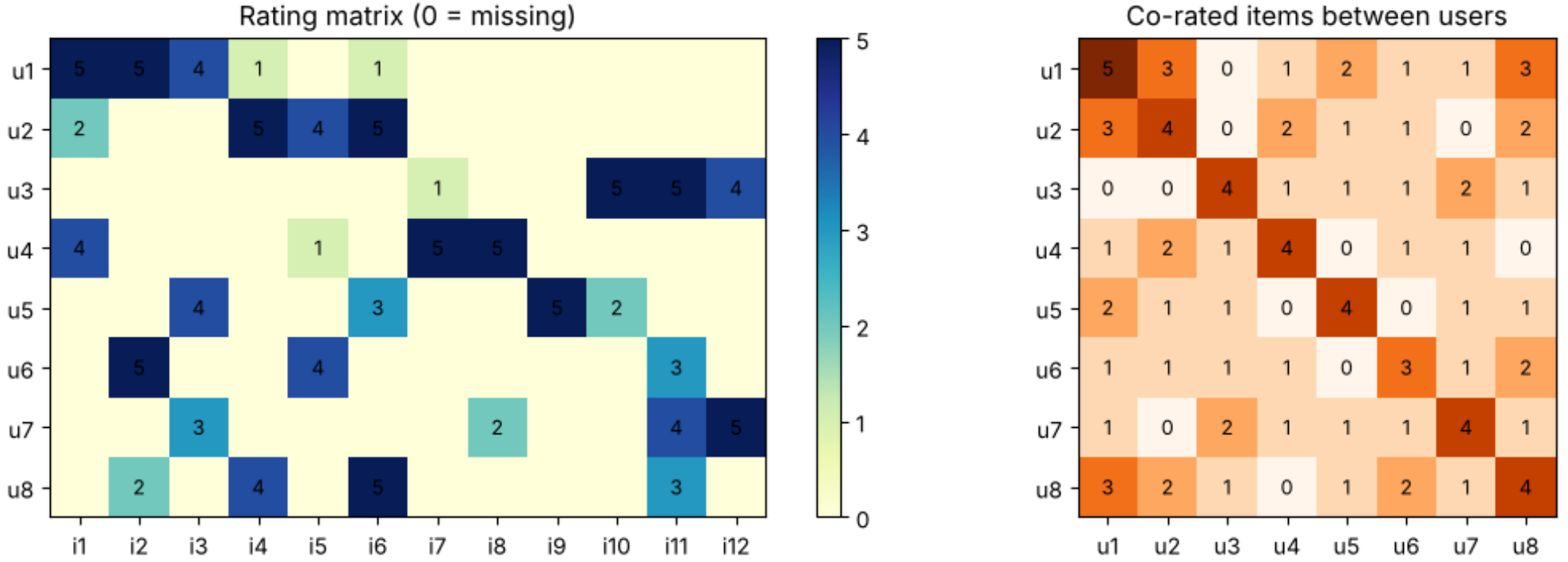
stats = pd.Series({
    "ความหนาแน่น (density)": known.sum() / R_FULL.size,
    "ค่าเฉลี่ยรวม (mu)": np.nanmean(R_FULL),
    "รายการร่วมเฉลี่ยต่อคู่ผู้ใช้": np.mean(pairs),
    "คู่ผู้ใช้ที่ไม่มีรายการร่วมเลย":
        int(np.sum(np.array(pairs) == 0)),
    "จำนวนคู่ชอบ (rating>=4)": int((np.nan_to_num(R_FULL) >=
        LIKE_T).sum()),
})
display(stats.round(4).to_frame("ค่า"))
display(pd.DataFrame({"ค่าเฉลี่ยผู้ใช้": np.nanmean(R_FULL,
    axis=1).round(2),
                      "จำนวนคะแนน": known.sum(axis=1)},
                         index=USERS).T)
display(pd.DataFrame({"จำนวนคะแนน": known.sum(axis=0),
                      "จำนวนผู้ชอบ": (np.nan_to_num(R_FULL)
                         >= LIKE_T).sum(axis=0)},
                         index=ITEMS).T)
```

อธิบายโค้ด

สี่ตัวเลขที่ต้องดูก่อนสร้างโมเดล

- **ความหนาแน่น** = 32/96 = 0.3333
- **รายการร่วม** ของผู้ใช้ทุกคู่ (co_rated) เฉลี่ย 1.11 เรื่อง และมี 6 คู่ที่ไม่มีรายการร่วมเลย
- ค่าเฉลี่ยรายผู้ใช้ (u1 = 3.20, u2/u6 = 4.00) และจำนวนคะแนน/จำนวนผู้ชอบต่อรายการ
- จำนวนคู่ชอบ = 20 ⟶ คือจำนวนกรณีของการทดลอง Ranking

Part 1: Heatmap เมทริกซ์คะแนนและรายการร่วม



- ซ้าย: เมทริกซ์คะแนน (0 = ว่าง) เห็นกลุ่มผู้ใช้แอนิเมชันและกลุ่ม Sci-Fi
- ขวา: รายการร่วมระหว่างผู้ใช้ — ส่วนใหญ่เป็น 0–1 เรื่อง
- ความหนาแน่น 33% แต่ **หลักฐานต่อคู่ยังน้อยมาก** ⟶ ความคล้ายเชื่อถือได้ต่ำ (5.4.2)

Part 2: ความคล้ายระหว่างผู้ใช้ (5.4)

```python
def pearson_users(R, a, b):
    """Pearson บนรายการร่วม
ใช้ค่าเฉลี่ยของผู้ใช้จากทุกคะแนนของผู้ใช้
        (ตามบทที่ 5)"""
    kn = ~np.isnan(R); co = kn[a] & kn[b]
    if co.sum() == 0: return 0.0, 0
    x = R[a, co] - np.nanmean(R[a]); y = R[b, co] -
        np.nanmean(R[b])
    den = np.sqrt((x**2).sum() * (y**2).sum())
    return (float((x*y).sum() / den) if den else 0.0),
        int(co.sum())

def cosine_users(R, a, b):
    x, y = np.nan_to_num(R[a]), np.nan_to_num(R[b])
    den = np.linalg.norm(x) * np.linalg.norm(y)
    return float(x @ y / den) if den else 0.0

def jaccard_users(R, a, b):
    kn = ~np.isnan(R)
    union = (kn[a] | kn[b]).sum()
    return float((kn[a] & kn[b]).sum() / union) if
        union else 0.0
```

อธิบายโค้ด

Pearson • Cosine • Jaccard

- Pearson: ลบค่าเฉลี่ยของผู้ใช้ (จากทุกคะแนน) แล้ววัดบนรายการร่วม เท่านั้น คืนค่า (s, n)
- Cosine บนคะแนนดิบ: ช่องว่าง = 0 จึงเป็นบวกเสมอ
- Jaccard: ดูว่าดูเรื่องเดียวกันไหม ไม่สนค่าคะแนน
- ผลกับ u1 ตรงตารางที่ 5.6: u1–u2 = −0.9086, u1–u5 = 0.9062

Part 2: Adjusted Cosine + การถ่วงความคล้าย

```python
def adjusted_cosine_items(R, i, j):
    """Adjusted Cosine (Sarwar et al.): ลบค่าเฉลี่ยของ
        'ผู้ใช้'ก่อนวัดมุมระหว่างรายการ"""
    kn = ~np.isnan(R); co = kn[:, i] & kn[:, j]
    if co.sum() == 0: return 0.0
    mu_u = np.nanmean(R, axis=1)
    x = R[co, i] - mu_u[co]; y = R[co, j] - mu_u[co]
    den = np.sqrt((x**2).sum() * (y**2).sum())
    return float((x*y).sum() / den) if den else 0.0

# Herlocker: s * min(n, beta)/beta
def significance_weight(s, n, beta=3):
    return s * min(n, beta) / beta

def shrink(s, n, lam=2):  # Koren: s * n/(n + lambda)
    return s * n / (n + lam)
```

อธิบายโค้ด

ความคล้ายระหว่างรายการ + ลดผลหลักฐานน้อย

- Adjusted Cosine (Sarwar): ลบค่าเฉลี่ยของ **ผู้ใช้** ก่อนวัดมุมระหว่างสองรายการ
- Significance weighting: $s \times \min(n, \beta)/\beta$ ($\beta = 3$)
- Shrinkage: $s \times n/(n + \lambda)$ ($\lambda = 2$)
- ความคล้าย +1 จากรายการร่วมเรื่องเดียวถูกลดเหลือ 0.3333 ทั้งสองวิธี (ตารางที่ 5.7)

Part 2: คลาสแม่ Recommender

```python
class Recommender:
    name = "Base"; family = "-"; rating_model = True
    def fit(self, R):
        self.R = R.copy(); self.kn = ~np.isnan(R)
        self.mu = float(np.nanmean(R))
        with np.errstate(all="ignore"):
            self.mu_u = np.where(self.kn.sum(1) > 0,
                np.nanmean(np.where(self.kn, R,
                np.nan), axis=1), self.mu)
        self._fit(); return self
    def _fit(self): pass
    # คะแนนทำนาย (ตัดให้อยู่ใน 1-5)
    def predict(self, u, i):
        return float(np.clip(self._predict(u, i), 1,
            5)) if self.rating_model else np.nan
    def _predict(self, u, i): raise NotImplementedError
    # คะแนนสำหรับจัดอันดับทุกรายการ
    def scores(self, u):
        return np.array([self.predict(u, i) for i in
            range(self.R.shape[1])])
    def explain(self, u, i): return ""
```

อธิบายโค้ด

interface เดียวกันทุกโมเดล

- `fit(R)`: เก็บ R, mask ช่องที่รู้, μ และค่าเฉลี่ยผู้ใช้ `mu_u` แล้วเรียก `_fit()` ของลูก
- `predict(u, i)`: ค่าทำนาย **ตัดให้อยู่ใน** 1–5 (โมเดล ranking คืน NaN)
- `scores(u)`: คะแนนจัดอันดับทุกรายการ ใช้กับการทดลอง Ranking
- `explain()` ให้ kNN รายงานเพื่อนบ้านที่ใช้ ⟶ นำไปแสดงใน GUI

Part 2: ระบบฐาน (Baseline)

```python
class GlobalMean(Recommender):
    name, family = "GlobalMean", "Baseline"
    def _predict(self, u, i): return self.mu

class UserMean(Recommender):
    name, family = "UserMean", "Baseline"
    def _predict(self, u, i): return self.mu_u[u]

class ItemMean(Recommender):
    name, family = "ItemMean", "Baseline"
    def _fit(self):
        cnt = self.kn.sum(0)
        with np.errstate(all="ignore"):
            self.mu_i = np.where(cnt > 0,
                np.nansum(np.nan_to_num(self.R), 0) /
                np.maximum(cnt, 1), self.mu)
    def _predict(self, u, i): return self.mu_i[i]

class Popularity(Recommender):
    """จำนวนผู้ชอบ (rating>=4) – ไม่ทำนายคะแนน
ใช้จัดอันดับอย่างเดียว"""
    name, family, rating_model = "Popularity", "Baseline", False
    def _fit(self): self.pop = (np.nan_to_num(self.R) >=
        LIKE_T).sum(0).astype(float)
    def scores(self, u): return self.pop.copy()
```

อธิบายโค้ด

ระบบฐานที่ต้องเทียบเสมอ

- GlobalMean: ทุกช่องได้ μ = 3.625
- UserMean / ItemMean: ค่าเฉลี่ยของผู้ใช้หรือของรายการ (ไม่มีคะแนน $\longrightarrow$ μ)
- Popularity: นับจำนวนผู้ชอบ (≥ 4) ไม่ทำนายคะแนน (`rating_model = False`) ใช้ ranking อย่างเดียว
- ไม่ปรับเฉพาะบุคคลเลย แต่เป็นคู่เทียบที่ขาดไม่ได้ (5.17.1)

Part 2: แบบจำลองค่าเอนเอียง (5.8)

```python
class BiasBaseline(Recommender):
    """สมการ 5.7: b_i ก่อนแล้ว b_u พร้อมการหด lam_i,
        lam_u"""
    name, family = "Baseline(bias)", "Model-based"
    def __init__(self, lam_i=2.0, lam_u=2.0):
        self.lam_i, self.lam_u = lam_i, lam_u
    def _fit(self):
        R, kn, mu = self.R, self.kn, self.mu
        self.b_i = np.array([(R[kn[:, i], i] -
            mu).sum() / (self.lam_i + kn[:, i].sum())
            for i in range(R.shape[1])])
        self.b_u = np.array([(R[u, kn[u]] - mu -
            self.b_i[kn[u]]).sum() / (self.lam_u +
            kn[u].sum()) for u in range(R.shape[0])])
    def _predict(self, u, i): return self.mu +
        self.b_u[u] + self.b_i[i]
```

อธิบายโค้ด

แยกผลของผู้ใช้และรายการ

- สมการ 5.7: คำนวณ b_i ก่อน แล้วใช้ b_i คำนวณ b_u
- ตัวหาร λ + n **หด** รายการที่มีคะแนนน้อยเข้าหาศูนย์ ($\lambda_i = \lambda_u = 2$)
- $\hat{r} = \mu + b_u + b_i$: ทุกคนได้ **ลำดับรายการเหมือนกัน** ต่างกันแค่ระดับ
- ผล u1: b_u1 = −0.306, b_i9 = 0.4583, ค่าทำนาย i9 = 3.7774 (ตารางที่ 5.11)

Part 2: User-kNN (สมการ 5.4)

```python
class UserKNN(Recommender):
    """สมการ 5.4: ยืมส่วนเบี่ยงเบนจากเพื่อนบ้านเชิงบวกไม่เกิน k คน"""
    name, family = "User-kNN", "Memory-based"
    def __init__(self, k=3, weighting=None, beta=3, lam=2, name=None):
        self.k, self.weighting, self.beta, self.lam = k, weighting, beta,
            lam
        if name: self.name = name
    def _fit(self):
        n = self.R.shape[0]; self.S = np.zeros((n, n))
        for a in range(n):
            for b in range(n):
                if a == b: continue
                s, c = pearson_users(self.R, a, b)
                if self.weighting == "significance": s =
                    significance_weight(s, c, self.beta)
                if self.weighting == "shrinkage": s = shrink(s, c, self.lam)
                self.S[a, b] = s
    def neighbors(self, u, i):
        nb = [(self.S[u, v], v) for v in range(self.R.shape[0]) if v != u
            and self.kn[v, i] and self.S[u, v] > 0]
        return sorted(nb, reverse=True)[:self.k]
    def _predict(self, u, i):
        nb = self.neighbors(u, i)
        if not nb: return self.mu_u[u]  # ไม่มีหลักฐาน → ค่าเฉลี่ยของผู้ใช้
        num = sum(s * (self.R[v, i] - self.mu_u[v]) for s, v in nb)
        return self.mu_u[u] + num / sum(abs(s) for s, v in nb)
    def explain(self, u, i):
        return ", ".join(f"{USERS[v]}({s:.2f})" for s, v in
            self.neighbors(u, i)) or "– (ใช้ค่าเฉลี่ยผู้ใช้)"
```

อธิบายโค้ด

ยืมส่วนเบี่ยงเบนจากเพื่อนบ้าน

- `_fit` สร้างเมทริกซ์ความคล้าย S ระหว่างผู้ใช้ (เลือก weighting ได้)
- `neighbors`: ผู้ใช้ที่ **ให้คะแนนรายการ i แล้ว** และความคล้าย **เป็นบวก** ไม่เกิน k คน
- ทำนาย = $\mu_u + \Sigma\, s\cdot(r_{vi} - \mu_v) / \Sigma|s|$
- ไม่มีเพื่อนบ้าน ⟶ คืน μ_u = "ไม่มีหลักฐาน" (สาเหตุของคะแนนเสมอ)
- u1: i9 = 4.70, i7 = i8 = 4.45 (ตารางที่ 5.8)

Part 2: Item-kNN (สมการ 5.6)

```python
class ItemKNN(Recommender):
    """สมการ 5.6:
        ค่าเฉลี่ยถ่วงน้ำหนักของคะแนนผู้ใช้เองต่อรายการที่คล้าย
เชิงบวก"""
    name, family = "Item-kNN", "Memory-based"
    def __init__(self, k=3): self.k = k
    def _fit(self):
        m = self.R.shape[1]; self.S = np.zeros((m, m))
        for i in range(m):
            for j in range(m):
                if i != j: self.S[i, j] =
                    adjusted_cosine_items(self.R, i, j)
    def neighbors(self, u, i):
        nb = [(self.S[i, j], j) for j in
            range(self.R.shape[1]) if j != i and self.kn[u,
            j] and self.S[i, j] > 0]
        return sorted(nb, reverse=True)[:self.k]
    def _predict(self, u, i):
        nb = self.neighbors(u, i)
        if not nb: return self.mu_u[u]
        return sum(s * self.R[u, j] for s, j in nb) / sum(s
            for s, j in nb)
    def explain(self, u, i):
        return ", ".join(f"{ITEMS[j]}({s:.2f})" for s, j in
            self.neighbors(u, i)) or "– (ใช้ค่าเฉลี่ยผู้ใช้)"
```

อธิบายโค้ด

ใช้คะแนนของผู้ใช้เองต่อเรื่องที่คล้าย

- S = Adjusted Cosine ระหว่างรายการทุกคู่ (12×12)
- เพื่อนบ้าน = รายการที่ **ผู้ใช้เคยให้คะแนน** และคล้ายเชิงบวก ไม่เกิน k เรื่อง
- ทำนาย = $\sum$ s•r_uj / $\sum$ s ⟶ อยู่ในช่วง 1–5 เสมอ
- u1: i7 = 5.0, i8 = 4.5, i9 = 4.0 ส่วน i5, i11, i12 ไม่มีหลักฐาน = 3.2 (ตารางที่ 5.9)

Part 2: User-kNN แบบข้อมูลโดยปริยาย (5.5)

```
class UserKNNImplicit(Recommender):
    """สมการ 5.5: Cosine บนเวกเตอร์ชอบ 0/1
แล้วรวมความคล้ายของเพื่อนบ้านที่ชอบรายการ"""
    name, family, rating_model =
        "User-kNN(implicit)", "Memory-based", False
    def __init__(self, k=3): self.k = k
    def _fit(self):
        X = (np.nan_to_num(self.R) >=
            LIKE_T).astype(float)
        nrm = np.linalg.norm(X, axis=1,
            keepdims=True); nrm[nrm == 0] = 1
        S = (X / nrm) @ (X / nrm).T;
            np.fill_diagonal(S, 0)
        self.X, self.S = X, S
    def scores(self, u):
        top = np.argsort(-self.S[u])[:self.k]
        return self.S[u, top] @ self.X[top]
```

อธิบายโค้ด

เมื่อมีเพียงชอบ / ไม่ได้ชอบ

- แปลงคะแนนเป็นเวกเตอร์ชอบ 0/1 (X)
- ความคล้าย = Cosine ของเวกเตอร์ชอบ (หารรากที่สองของจำนวนเรื่องที่ชอบ)
- คะแนนรายการ = ผลรวมความคล้ายของเพื่อนบ้าน k คนที่ชอบรายการนั้น
- u1: i5 = i9 = 0.4082 > i7 = i8 = 0.3333 (ตัวอย่างการคำนวณ 5.4) — i5 ขึ้นมาเพราะไม่เห็นว่า u4 เกลียด Wicked

Part 2: MF ด้วย SGD (5.9)

```python
class MF_SGD(Recommender):
    """สมการ 5.8-5.9: MF + bias ฝึกด้วย SGD"""
    name, family = "MF-SGD", "Latent factor"
    def __init__(self, k=2, lr=0.05, reg=0.5, epochs=150, seed=SEED):
        self.k, self.lr, self.reg, self.epochs, self.seed = k, lr, reg,
            epochs, seed
    def _fit(self):
        rng = np.random.default_rng(self.seed); nU, nI = self.R.shape
        self.P = rng.normal(0, 0.1, (nU, self.k)); self.Q =
            rng.normal(0, 0.1, (nI, self.k))
        self.bu = np.zeros(nU); self.bi = np.zeros(nI)
        obs = list(zip(*np.where(self.kn))); self.history = []
        for ep in range(self.epochs):
            for t in rng.permutation(len(obs)):
                u, i = obs[t]; r = self.R[u, i]
                e = r - (self.mu + self.bu[u] + self.bi[i] + self.P[u]
                    @ self.Q[i])
                self.bu[u] += self.lr * (e - self.reg * self.bu[u])
                self.bi[i] += self.lr * (e - self.reg * self.bi[i])
                pu = self.P[u].copy()
                self.P[u] += self.lr * (e * self.Q[i] - self.reg *
                    self.P[u])
                self.Q[i] += self.lr * (e * pu - self.reg * self.Q[i])
            self.history.append(np.sqrt(np.mean([(self.R[u, i] -
                self._predict(u, i))**2 for u, i in obs])))
    def _predict(self, u, i): return self.mu + self.bu[u] + self.bi[i]
        + self.P[u] @ self.Q[i]
```

อธิบายโค้ด

ปัจจัยแฝง + ค่าเอนเอียง

- $\hat{r} = \mu + b_u + b_i + p_u \cdot q_i$ ฝึกเฉพาะ **ช่องที่รู้ค่า** (obs) ในลำดับสุ่ม
- กฎปรับ (5.9): พารามิเตอร์ += lr × (e × ตัวคูณ − reg × พารามิเตอร์)
- `pu = self.P[u].copy()` ใช้ค่าเดิมปรับ Q ⟶ ไม่ใช้ค่าที่เพิ่งเปลี่ยน
- `history` เก็บ RMSE ทุกรอบ ⟶ วาดกราฟการลู่เข้าใน Part 10

Part 2: ALS — สลับแก้ทีละฝั่ง (5.10)

```python
class ALS_Explicit(Recommender):
    """สมการ 5.10: สลับแก้ P และ Q ด้วยคำตอบรูปปิดบนคะแนนที่หัก
        mu (ไม่มี bias)"""
    name, family = "ALS", "Latent factor"
    def __init__(self, k=2, lam=0.5, iters=15, seed=SEED):
        self.k, self.lam, self.iters, self.seed = k, lam, iters,
        seed
    def _fit(self):
        rng = np.random.default_rng(self.seed); nU, nI =
            self.R.shape
        self.P = rng.normal(0, 0.1, (nU, self.k)); self.Q =
            rng.normal(0, 0.1, (nI, self.k))
        Rc = self.R - self.mu; I = np.eye(self.k)
        for _ in range(self.iters):
            for u in range(nU):
                idx = self.kn[u]
                if idx.any(): self.P[u] =
                    np.linalg.solve(self.Q[idx].T @ self.Q[idx] +
                    self.lam*I, self.Q[idx].T @ Rc[u, idx])
            for i in range(nI):
                idx = self.kn[:, i]
                if idx.any(): self.Q[i] =
                    np.linalg.solve(self.P[idx].T @ self.P[idx] +
                    self.lam*I, self.P[idx].T @ Rc[idx, i])
    def _predict(self, u, i): return self.mu + self.P[u] @
        self.Q[i]
```

อธิบายโค้ด

คำตอบรูปปิดแทน SGD

- ตรึง Q แล้วแก้ $p_u = (Q^TQ + \lambda I)^{-1} Q^T(r - \mu)$ ของผู้ใช้แต่ละคน
- จากนั้นตรึง P แล้วแก้ q_i ของทุกรายการ ทำซ้ำ **iters** รอบ
- ไม่ต้องเลือก learning rate และแต่ละผู้ใช้แก้แยกกันได้ (ขนานได้)
- รุ่นนี้ไม่มีค่าเอนเอียง ใช้คะแนนที่หัก μ แล้ว (ตามตัวอย่างในบท)

Part 2: WRMF — ข้อมูลโดยปริยาย (5.11)

```python
class WRMF(Recommender):
    """สมการ 5.11: ความชอบ p=1[r>=4], ความเชื่อมั่น c = 1 +
        alpha*r/5 (ช่องว่าง c=1) ฝึกด้วย ALS"""
    name, family, rating_model = "WRMF", "Implicit", False
    def __init__(self, k=2, alpha=10, lam=0.1, iters=15,
        seed=SEED):
        self.k, self.alpha, self.lam, self.iters, self.seed =
            k, alpha, lam, iters, seed
    def _fit(self):
        Pm = (np.nan_to_num(self.R) >= LIKE_T).astype(float)
        Cm = 1 + self.alpha * np.nan_to_num(self.R) / 5
        rng = np.random.default_rng(self.seed); nU, nI =
            self.R.shape
        X = rng.normal(0, 0.1, (nU, self.k)); Y = rng.normal(0,
            0.1, (nI, self.k)); I = np.eye(self.k)
        for _ in range(self.iters):
            for u in range(nU):
                Cu = np.diag(Cm[u]); X[u] = np.linalg.solve(Y.T
                    @ Cu @ Y + self.lam*I, Y.T @ Cu @ Pm[u])
            for i in range(nI):
                Ci = np.diag(Cm[:, i]); Y[i] =
                    np.linalg.solve(X.T @ Ci @ X + self.lam*I,
                    X.T @ Ci @ Pm[:, i])
        self.S = X @ Y.T
    def scores(self, u): return self.S[u].copy()
```

อธิบายโค้ด

แยกความชอบออกจากความเชื่อมั่น

- ความชอบ $p = 1[r \geq 4]$ • ความเชื่อมั่น $c = 1 + \alpha \cdot r/5$ (ช่องว่าง $c = 1$)
- คะแนน 1 จึงเป็นตัวอย่างลบที่เชื่อมั่นปานกลาง ($c = 3$) ต่างจากช่องว่าง
- ทุกช่องมีส่วนในค่าสูญเสีย ⟶ ฝึกด้วย ALS ที่ถ่วงด้วย diag(c)
- u1: i9 0.8217, i8 0.7671, i7 0.7644 ตรงกับบท

Part 2: BPR และ EASE (5.12)

```python
class BPR(Recommender):
    """สมการ 5.12: เรียนลำดับจากชุดสาม (u, i ชอบ, j ไม่ได้ชอบ) ด้วย
        SGD"""
    name, family, rating_model = "BPR", "Implicit", False
    def __init__(self, k=2, lr=0.05, reg=0.01, epochs=400, seed=SEED):
        self.k, self.lr, self.reg, self.epochs, self.seed = k, lr, reg,
            epochs, seed
    def _fit(self):
        rng = np.random.default_rng(self.seed); nU, nI = self.R.shape
        X = (np.nan_to_num(self.R) >= LIKE_T)
        P = rng.normal(0, 0.1, (nU, self.k)); Q = rng.normal(0, 0.1,
            (nI, self.k))
        pos = [(u, i) for u in range(nU) for i in range(nI) if X[u, i]]
        for _ in range(self.epochs):
            for t in rng.permutation(len(pos)):
                u, i = pos[t]
                neg = np.where(~X[u])[0]
                if len(neg) == 0: continue
                j = rng.choice(neg)
                # ขนาดของความผิด
                x = P[u] @ (Q[i] - Q[j]); g = 1 / (1 + np.exp(x))
                pu = P[u].copy()
                P[u] += self.lr * (g * (Q[i] - Q[j]) - self.reg * P[u])
                Q[i] += self.lr * (g * pu - self.reg * Q[i])
                Q[j] += self.lr * (-g * pu - self.reg * Q[j])
        self.S = P @ Q.T
    def scores(self, u): return self.S[u].copy()
```

อธิบายโค้ด

โมเดลเพื่อการจัดอันดับ

- BPR: สุ่มชุดสาม (u, i ชอบ, j ไม่ได้ชอบ) แล้วเพิ่ม $\sigma(\hat{x}_{ui} - \hat{x}_{uj})$
- $g = 1/(1+e^{x})$ คือขนาดความผิด: ยิ่งจัดลำดับผิดยิ่งขยับมาก
- EASE: $P = (X^{T}X + \lambda I)^{-1}$, $B = -P/\mathrm{diag}(P)$, $\mathrm{diag}(B) = 0$ — คำตอบรูปปิด 4 บรรทัด
- EASE u1: i9 0.1102, i5 0.1101 (สอดคล้องโปรแกรมที่ 5.10)

Part 2: รายชื่อโมเดลที่เปรียบเทียบ

```python
def make_models():
    """โมเดลทั้งหมดที่ใช้เปรียบเทียบ
        (ค่าไฮเพอร์พารามิเตอร์ตามตารางที่ 5.20)"""
    return [GlobalMean(), UserMean(), ItemMean(),
        Popularity(), BiasBaseline(2, 2),
            UserKNN(k=3), UserKNN(k=3,
                weighting="significance",
                name="User-kNN+SigW"),
            ItemKNN(k=3), UserKNNImplicit(k=3),
            MF_SGD(k=2, lr=0.05, reg=0.5, epochs=150),
                ALS_Explicit(k=2, lam=0.5),
            WRMF(k=2, alpha=10, lam=0.1), BPR(k=2,
                lr=0.05, reg=0.01, epochs=400),
                EASE(lam=5.0)]

MODEL_NAMES = [m.name for m in make_models()]
RATING_MODELS = [m.name for m in make_models() if
    m.rating_model]
def get_model(name):
    return [m for m in make_models() if m.name ==
        name][0]
```

อธิบายโค้ด

Model registry

- `make_models()` สร้างโมเดลใหม่ทุกครั้ง ⟶ ทุก fold เริ่มจากศูนย์ ไม่มีสถานะค้าง
- ไฮเพอร์พารามิเตอร์ตามตารางที่ 5.20: k = 3, λ = 2, MF k = 2 / reg 0.5, WRMF α = 10, EASE λ = 5
- `RATING_MODELS` = 9 โมเดลที่ทำนายคะแนนได้ ใช้ใน Exp.1
- เพิ่มโมเดลใหม่: เขียนคลาสลูก แล้วใส่ในรายการนี้ที่เดียว

Part 2: Metric ของ Rating และ Ranking

```python
def rmse(y, p): y, p = np.asarray(y), np.asarray(p); return
    float(np.sqrt(np.mean((y - p)**2)))
def mae(y, p):  y, p = np.asarray(y), np.asarray(p); return
    float(np.mean(np.abs(y - p)))

# ---------- 2.4 Accuracy Metrics: Ranking (list-based, เหมือนต้นแบบบทที่
    3) ----------
def precision_at_k(recs, rel, k): return len(set(recs[:k]) & set(rel)) / k
    if k else 0.0
def recall_at_k(recs, rel, k): rel = set(rel); return len(set(recs[:k]) &
    rel) / len(rel) if rel else 0.0
def f1_at_k(recs, rel, k):
    p, r = precision_at_k(recs, rel, k), recall_at_k(recs, rel, k); return
        0.0 if p + r == 0 else 2*p*r/(p+r)
def hr_at_k(recs, rel, k): return float(bool(set(recs[:k]) & set(rel)))
def reciprocal_rank(recs, rel):
    for rank, iid in enumerate(recs, 1):
        if iid in set(rel): return 1.0 / rank
    return 0.0
def ap_at_k(recs, rel, k):
    rel = set(rel); hits = 0; s = 0.0
    for n, iid in enumerate(recs[:k], 1):
        if iid in rel: hits += 1; s += hits / n
    return s / min(len(rel), k) if rel else 0.0
def ndcg_at_k(recs, rel, k):
    rel = set(rel)
    dcg = sum(1/math.log2(n+2) for n, iid in enumerate(recs[:k]) if iid in
        rel)
    idcg = sum(1/math.log2(n+2) for n in range(min(len(rel), k)))
    return dcg / idcg if idcg else 0.0
```

อธิบายโค้ด

ชุดเดียวกับต้นแบบบทที่ 3

- RMSE / MAE สำหรับค่าทำนายคะแนน
- P@K, R@K, F1@K, HR@K, MRR, AP@K, nDCG@K แบบรายการ ใช้ใน Part 8 และ GUI
- `rel` เป็นเซตของรายการที่ผู้ใช้ชอบจริง
- nDCG ใช้ส่วนลด 1/log2(อันดับ + 1)

Part 2: Ranking เมื่อมีคะแนนเสมอ (สมการ 5.14)

```python
def tie_aware_metrics(score_vec, cand, target, k=3):
    """คืน metric ทั้งแบบ 'ค่าคาดหมาย' (สุ่มลำดับรายการที่เสมอกัน)
และแบบเรียงตามรหัส"""
    s = np.round(np.asarray(score_vec)[cand], 10); st =
        round(float(score_vec[target]), 10)
    # ties รวมตัวมันเอง
    higher = int((s > st).sum()); ties = int((s == st).sum())
    # อันดับที่เป็นไปได้ทั้งหมด
    ranks = higher + np.arange(1, ties + 1)
    hr = float(np.mean(ranks <= k)); rr = float(np.mean(1 / ranks))
    ndcg = float(np.mean([1/math.log2(r+1) if r <= k else 0.0 for r
        in ranks]))
    ap = float(np.mean([1/r if r <= k else 0.0 for r in ranks]))
    auc = ((s < st).sum() + 0.5*(ties - 1)) / max(len(cand) - 1, 1)
    # เรียงเสมอตามรหัส
    order_id = sorted(cand, key=lambda j:
        (-round(float(score_vec[j]), 10), j))
    rank_id = order_id.index(target) + 1
    return {f"HR@{k}": hr, f"Precision@{k}": hr / k, f"Recall@{k}":
        hr,
            f"F1@{k}": (2*(hr/k)*hr/((hr/k)+hr)) if hr else 0.0,
            "MRR": rr, f"MAP@{k}": ap, f"nDCG@{k}": ndcg, "AUC":
                float(auc),
            f"HR@{k}(id-order)": float(rank_id <= k),
                "MRR(id-order)": 1/rank_id, "tie": ties > 1}
```

อธิบายโค้ด

ค่าคาดหมายแทนการเรียงตามรหัส

- `higher` = จำนวนรายการที่ได้คะแนน **สูงกว่า** เรื่องที่ซ่อน, `ties` = จำนวนที่ **เท่ากัน** (รวมตัวเอง)
- อันดับที่เป็นไปได้ = higher + 1 ... higher + ties → เฉลี่ย HR, RR, nDCG, AP
- AUC นับคู่ที่เสมอเป็น 0.5
- คืนค่าแบบ id-order คู่กันเพื่อแสดงว่ากติกาคะแนนเสมอเปลี่ยนผลได้

Part 2: Top-K + Beyond-Accuracy (1/2)

```python
def topk_random_tiebreak(score_vec, cand, k, rng):
    """Top-K ที่สุ่มลำดับรายการเสมอ (ใช้สำหรับ Beyond-Accuracy)"""
    cand = np.asarray(cand); s =
        np.round(np.asarray(score_vec)[cand], 10)
    order = np.lexsort((rng.random(len(cand)), -s))
    return [int(j) for j in cand[order][:k]]

# ---------- 2.6 Beyond-Accuracy Metrics ----------
def coverage(all_recs): return len(set(all_recs)) / len(ITEMS)
def gini_coefficient(all_recs):
    counts = pd.Series(all_recs).value_counts()
    x = np.sort(np.array([counts.get(j, 0) for j in
        range(len(ITEMS))], dtype=float)); n = len(x)
    if x.sum() == 0: return 0.0
    return float((2*np.sum(np.arange(1, n+1)*x)/(n*x.sum())) -
        (n+1)/n)
def ild(rec_idx):
    """Intra-List Diversity จากเวกเตอร์แนว (1 - cosine)"""
    if len(rec_idx) < 2: return 0.0
    V = G_MAT[rec_idx]; V = V / np.maximum(np.linalg.norm(V,
        axis=1, keepdims=True), 1e-12)
    D = 1 - V @ V.T; return float(D[np.triu_indices(len(rec_idx),
        1)].mean())
def novelty(rec_idx, R_train):
    pop = (~np.isnan(R_train)).sum(0); n_users = R_train.shape[0]
    return float(np.mean([-math.log2(max(1, pop[j]) / n_users) for
        j in rec_idx])) if rec_idx else 0.0
```

อธิบายโค้ด

วัดมากกว่าความแม่น

- `topk_random_tiebreak`: เรียงคะแนนจากมากไปน้อย สุ่มลำดับเฉพาะรายการที่เสมอ (`lexsort`)
- Coverage: สัดส่วนรายการในคลังที่เคยถูกแนะนำ • Gini: ความเหลื่อมล้ำ (ต่ำ = กระจาย)
- ILD: 1 − cosine ของเวกเตอร์แนวภายในรายการ
- Novelty: −log2(จำนวนผู้ให้คะแนน / จำนวนผู้ใช้) ยิ่งสูงยิ่งไม่ดัง

Part 2: Top-K + Beyond-Accuracy (2/2)

```python
def calibration_genre(u, rec_idx, R_train):
    hist = np.where(~np.isnan(R_train[u]))[0]
    hp = G_MAT[hist].sum(0); rp = G_MAT[rec_idx].sum(0)
    if hp.sum() == 0 or rp.sum() == 0: return np.nan
    return float(1 - 0.5*np.abs(hp/hp.sum() -
        rp/rp.sum()).sum())
def personalization(list_of_recs):
    """1 - ค่าเฉลี่ยความซ้ำของรายการแนะนำระหว่างผู้ใช้แต่ละคู่
        """
    L = [set(x) for x in list_of_recs]
    if len(L) < 2: return np.nan
    ov = [len(a & b) / max(len(a), 1) for a, b in
        itertools.combinations(L, 2)]
    return float(1 - np.mean(ov))
# Average Recommendation Popularity
def arp(rec_idx, R_train):
    pop = (~np.isnan(R_train)).sum(0); return
        float(np.mean(pop[rec_idx])) if rec_idx else 0.0

def candidates_for(R_train, u):
    """รายการผู้สมัคร =
เรื่องที่ผู้ใช้ยังไม่มีคะแนนในข้อมูลฝึก (Full
        Catalog)"""
    return [j for j in range(R_train.shape[1]) if
        np.isnan(R_train[u, j])]
```

อธิบายโค้ด

ความสอดคล้องและความเป็นส่วนตัว

- Calibration = 1 – ระยะ Total Variation ระหว่างสัดส่วนแนวในประวัติกับในรายการแนะนำ
- Personalization = 1 – ค่าเฉลี่ยความซ้ำของรายการแนะนำระหว่างผู้ใช้ทุกคู่
- ARP: ความนิยมเฉลี่ยของรายการที่แนะนำ
- `candidates_for`: Full Catalog = เรื่องที่ยังไม่มีคะแนนในข้อมูลฝึก

Part 3: ตัวอย่าง u1 — ความคล้ายทุกมาตรวัด

```
u1 = U_INDEX[USERS[0]]
rows = []
for v in range(len(USERS)):
    if v == u1: continue
    s, n = pearson_users(R_FULL, u1, v)
    rows.append({"pair": f"{USERS[u1]}-{USERS[v]}",
        "n_co": n, "Pearson": s,
                 "SigWeight(β=3)":
                     significance_weight(s, n, 3),
                     "Shrink(λ=2)": shrink(s, n, 2),
                 "Cosine": cosine_users(R_FULL, u1, v),
                     "Jaccard": jaccard_users(R_FULL,
                     u1, v)})
display(pd.DataFrame(rows).round(4))
```

อธิบายโค้ด

ทวนตารางที่ 5.6–5.7 ด้วย code

- วนผู้ใช้ทุกคนเทียบกับ u1: n รายการร่วม, Pearson, ถ่วงนัยสำคัญ, หด, Cosine, Jaccard
- u1–u4, u1–u6 ได้ Pearson = 1.0 จากหลักฐาน **เรื่องเดียว** ⟶ ถ่วงแล้วเหลือ 0.3333
- Cosine เป็นบวกทุกคู่, Jaccard ให้ u2/u8 สูงสุดทั้งที่รสนิยมตรงข้าม u1

Part 3: ค่าทำนายของ u1 จากทุกโมเดล (ผลจริงจาก Notebook)

โมเดล	i5	i7	i8	i9	i10	i11	i12
User-kNN	1.825	4.450	4.450	**4.700**	1.700	2.200	3.200
Item-kNN	3.200	**5.000**	4.500	4.000	1.000	3.200	3.200
Baseline(bias)	2.944	3.007	3.257	**3.777**	3.257	3.402	3.757
MF-SGD	2.731	3.229	3.414	**4.514**	2.642	3.100	3.803
WRMF (score)	0.422	0.764	0.767	**0.822**	−0.167	−0.509	−0.267
EASE (score)	0.1101	0.0944	0.0944	**0.1102**	0.0000	0.0000	0.0000
Popularity (#likes)	2	1	1	1	1	2	2

ทุกวิธีเชิงพฤติกรรมให้ i9 Conclave สูง เพราะ u5 ชอบทั้ง The Wild Robot และ Conclave — เชื่อมข้ามแนวได้

Part 4: Protocol Card ของการทดลองแบบ Offline

ประเด็น	Exp.1 Rating	Exp.2 Ranking	Validation (Part 8)
แบ่งข้อมูล	Leave-One-Rating-Out (32)	Leave-One-Liked-Out (20)	สุ่ม 25% ต่อผู้ใช้ (seed 42)
ฝึกใหม่	ทุก fold	ทุก fold	ครั้งเดียวบนชุดฝึก
Candidate	—	เรื่องที่ยังไม่มีคะแนน + เรื่องที่ซ่อน	เรื่องที่ไม่มีในชุดฝึก
Metric	RMSE, MAE (ตัด 1–5)	P/R/F1/HR@3, MRR, MAP, nDCG, AUC	RMSE/MAE + P/R/nDCG@3
คะแนนเสมอ	—	ค่าคาดหมาย + id-order	สุ่มลำดับ (seed)
Beyond-Accuracy	—	Coverage, Gini, ILD, Novelty, Calibration, Personalization, ARP	Coverage

ระบุทุกกติกาที่มีผลต่อตัวเลข ⟶ ทำซ้ำได้และเทียบกันได้อย่างยุติธรรม (หัวข้อ 5.14 และ 5.17.2)

Part 4: Leave-One-Out folds

```
def leave_one_rating_out_folds(R):
    """แต่ละ fold ซ่อนคะแนน 1 ค่า → (u, i, ค่าจริง,
        R_train)"""
    folds = []
    for u, i in zip(*np.where(~np.isnan(R))):
        Rt = R.copy(); Rt[u, i] = np.nan;
            folds.append((u, i, R[u, i], Rt))
    return folds

def leave_one_liked_out_folds(R, min_likes=2):
    """เหมือนบทที่ 3-4: ผู้ใช้ที่ชอบ >= 2เรื่อง
ซ่อนเรื่องที่ชอบทีละเรื่อง"""
    folds = []
    for u in range(R.shape[0]):
        likes = [i for i in range(R.shape[1]) if not
            np.isnan(R[u, i]) and R[u, i] >= LIKE_T]
        if len(likes) < min_likes: continue
        for h in likes:
            Rt = R.copy(); Rt[u, h] = np.nan;
                folds.append((u, h, Rt))
    return folds
```

อธิบายโค้ด

สองวิธีเลือกกลุ่มตัวอย่างแบบ LOO

- LOO rating: ซ่อนคะแนนทีละค่า ได้ 32 fold สำหรับ Exp.1
- LOO liked: ผู้ใช้ที่ชอบ ≥ 2 เรื่อง ซ่อนเรื่องที่ชอบทีละเรื่อง ได้ 20 fold (ชุดเดียวกับบทที่ 3–4)
- ทุก fold คัดลอก R ก่อนแก้ จึงไม่รั่วไปยัง fold อื่น
- ทุกโมเดลใช้ folds ชุดเดียวกัน ⟶ เปรียบเทียบแบบจับคู่ได้

Part 4: Validation split

```
def validation_split(R, frac=0.25, seed=42):
    """สุ่มกันคะแนนต่อผู้ใช้ไว้เป็น Validation (อย่างน้อย 1
ค่าและเหลือในชุดฝึกอย่างน้อย 2ค่า)"""
    rng = np.random.default_rng(seed); Rtr = R.copy(); val =
        []
    for u in range(R.shape[0]):
        rated = np.where(~np.isnan(R[u]))[0]
        n_val = min(max(1, int(round(frac * len(rated)))),
            max(len(rated) - 2, 0))
        for i in rng.choice(rated, size=n_val, replace=False):
            val.append((u, int(i), R[u, i])); Rtr[u, i] =
                np.nan
    return Rtr, pd.DataFrame(val, columns=["u", "i",
        "rating"])

RATING_FOLDS = leave_one_rating_out_folds(R_FULL)
RANK_FOLDS = leave_one_liked_out_folds(R_FULL)
R_TRAIN_VAL, VAL = validation_split(R_FULL, frac=0.25,
    seed=42)
print(f"Rating folds = {len(RATING_FOLDS)} | Ranking folds =
    {len(RANK_FOLDS)} | Validation ratings = {len(VAL)}")
display(VAL.assign(user=VAL.u.map(lambda x: USERS[x]),
    item=VAL.i.map(lambda x: f"{ITEMS[x]}
    {TITLE[ITEMS[x]]}"))[["user","item","rating"]])
```

อธิบายโค้ด

ชุดตรวจสอบสำหรับดู Output

- สุ่ม 25% ของคะแนนแต่ละผู้ใช้ (อย่างน้อย 1 ค่า) ด้วย seed 42
- บังคับให้เหลือในชุดฝึกอย่างน้อย 2 ค่า → kNN ยังหาเพื่อนบ้านได้
- ได้ 8 คะแนน: u1–i1 (5), u2–i6 (5), u3–i11 (5), u4–i5 (1) ...
- ใช้ทั้งตัวอย่าง Output ใน Part 8 และโหมด Validation ใน GUI

Part 5: Exp.1 Rating (Leave-One-Rating-Out)

```
def evaluate_rating(model_factory, folds=None):
    folds = folds or RATING_FOLDS; y, p = [], []
    for u, i, r, Rt in folds:
        m = model_factory().fit(Rt); y.append(r);
            p.append(m.predict(u, i))
    return {"RMSE": rmse(y, p), "MAE": mae(y, p)},
        np.array(p)

t0 = time.time(); rating_rows = []; LOO_PRED = {}
for name in RATING_MODELS:
    res, preds = evaluate_rating(lambda name=name:
        get_model(name))
    rating_rows.append({"Model": name, "Family":
        get_model(name).family, **res});
        LOO_PRED[name] = preds
rating_results =
    pd.DataFrame(rating_rows).sort_values("RMSE").rese
    t_index(drop=True)
display(rating_results.round(4))
print(f"ใช้เวลา {time.time()-t0:.1f} s")
```

อธิบายโค้ด

ฝึกใหม่ 32 ครั้งต่อโมเดล

- `model_factory` สร้างโมเดลใหม่ทุก fold → โมเดลไม่เคยเห็นคะแนนที่ซ่อน
- เก็บค่าจริง/ค่าทำนายทุก fold แล้วคิด RMSE, MAE
- `LOO_PRED` เก็บค่าทำนายรายกรณีไว้วิเคราะห์ต่อ
- ใช้เวลาราว 2 วินาทีสำหรับ 9 โมเดล

Part 5: ผล Exp.1 — RMSE / MAE แบบ LOO 32 ครั้ง

โมเดล	ตระกูล	RMSE	MAE
ALS	Latent factor	1.2712	0.9833
GlobalMean	Baseline	1.4541	1.2581
Item-kNN	Memory-based	1.6331	1.2969
UserMean	Baseline	1.8226	1.5625
User-kNN / +SigW	Memory-based	1.8534	1.5234
Baseline(bias)	Model-based	1.9531	1.7139
MF-SGD	Latent factor	1.9803	1.7105
ItemMean	Baseline	2.2006	1.9194

ค่าตรงตารางที่ 5.16: ในวิธีที่บทเทียบ GlobalMean ชนะ — ข้อมูล 32 คะแนนทำให้โมเดลที่ซับซ้อนเรียนรู้เกิน

Part 6: Exp.2 Ranking — evaluate_ranking

```python
def evaluate_ranking(model_factory, folds=None, k=K, seed=SEED):
    folds = folds or RANK_FOLDS; rng =
        np.random.default_rng(seed)
    rows, all_recs, lists = [], [], []
    for u, h, Rt in folds:
        m = model_factory().fit(Rt); sc = m.scores(u); cand =
            candidates_for(Rt, u)
        met = tie_aware_metrics(sc, cand, h, k)
        top = topk_random_tiebreak(sc, cand, k, rng); all_recs
            += top; lists.append(top)
        rows.append({"user": USERS[u], "heldout": ITEMS[h],
            **met,
                     "ILD": ild(top), "Novelty": novelty(top,
                         Rt), "Calibration":
                         calibration_genre(u, top, Rt),
                     "ARP": arp(top, Rt), "TopK": " |
                         ".join(ITEMS[j] for j in top)})
    detail = pd.DataFrame(rows)
    summary = detail.select_dtypes(include=[np.number]).mean().t
        o_dict()
    summary["tie cases"] =
        f"{int(detail['tie'].sum())}/{len(detail)}"
    summary["Coverage"] = coverage(all_recs); summary["Gini"] =
        gini_coefficient(all_recs)
    summary["Personalization"] = personalization(lists)
    return pd.Series(summary), detail
```

อธิบายโค้ด

หนึ่งฟังก์ชันได้ทั้ง Accuracy และ Beyond

- แต่ละ fold: ฝึกใหม่ → `scores(u)` → ผู้สมัคร Full Catalog → `tie_aware_metrics`
- Top-3 แบบสุ่มลำดับเสมอ → ILD, Novelty, Calibration, ARP รายกรณี
- สรุประดับระบบ: Coverage, Gini, Personalization และจำนวนกรณีที่มีคะแนนเสมอ
- คืน `summary` (ค่าเฉลี่ย) และ `detail` (รายกรณี) สำหรับ bootstrap

Part 6: ผล Exp.2 — Ranking บน 20 กรณี (ค่าคาดหมาย)

โมเดล	HR@3	MRR	nDCG@3	AUC	HR@3 (id)	MRR (id)	เสมอ
User-kNN	0.5662	0.5164	0.4692	0.6911	0.75	0.5621	10/20
EASE	0.4912	0.5117	0.4412	0.6437	0.60	0.5381	14/20
Item-kNN	0.5265	0.4670	0.4201	0.6538	0.65	0.4655	16/20
User-kNN(implicit)	0.5089	0.4620	0.4131	0.6228	0.60	0.4577	18/20
ALS	0.5000	0.4144	0.3604	0.6749	0.50	0.4269	1/20
BPR	0.4500	0.3710	0.3131	0.6282	0.45	0.3710	0/20
MF-SGD	0.4000	0.3296	0.2815	0.4872	0.40	0.3296	0/20
WRMF	0.2000	0.2577	0.1315	0.4751	0.20	0.2577	0/20
Popularity	0.0000	0.1379	0.0000	0.1713	0.00	0.1629	16/20

User-kNN / Item-kNN / EASE / Popularity ตรงตารางที่ 5.17 • id-order ต่างจากค่าคาดหมายมากในโมเดลที่มีคะแนนเสมอบ่อย

Part 7: Exp.3 Beyond-Accuracy

```
BEYOND_COLS = ["Coverage", "Gini", "ILD", "Novelty",
    "Calibration", "Personalization", "ARP"]
display(ranking_results[["Model", "Family"] +
    BEYOND_COLS].sort_values("Coverage",
    ascending=False).round(4).reset_index(drop=True))
```

อธิบายโค้ด

อ่านคู่กับความแม่นเสมอ

- ใช้ผลจาก `evaluate_ranking` ชุดเดิม ไม่ต้องรันใหม่
- Popularity: Coverage 0.67, Novelty ต่ำสุด 1.40, ARP สูงสุด 3.10 → แนะนำแต่เรื่องดัง
- ItemMean / Baseline(bias): Coverage 0.50, Gini 0.68 → ทุกคนได้ลำดับเหมือนกัน
- โมเดลเชิงพฤติกรรมส่วนใหญ่ Coverage 1.0 เพราะคลังมีเพียง 12 เรื่อง

Part 8: Validation — ค่าทำนายคะแนนเทียบค่าจริง

```
fitted_val = {m.name: m.fit(R_TRAIN_VAL) for m in
    make_models()}

# 8.1 Rating: ค่าทำนายเทียบค่าจริง
val_rating = VAL.copy()
val_rating.insert(0, "user", val_rating.u.map(lambda x:
    USERS[x]))
val_rating.insert(1, "movie", val_rating.i.map(lambda
    x: f"{ITEMS[x]} {TITLE[ITEMS[x]]}"))
for name in RATING_MODELS:
    val_rating[name] = [fitted_val[name].predict(u, i)
        for u, i in zip(VAL.u, VAL.i)]
print("8.1 Validation — Rating prediction (ค่าจริง vs
ค่าทำนาย)")
display(val_rating.drop(columns=["u", "i"]).round(2))
val_rating_metrics = pd.DataFrame({name: {"RMSE":
    rmse(VAL.rating, val_rating[name]), "MAE":
    mae(VAL.rating, val_rating[name])}
                                   for name in
                                    RATING_MODELS}).T.so
                                    rt_values("RMSE")
display(val_rating_metrics.round(4).T)
```

อธิบายโค้ด

ฝึกครั้งเดียวบนชุดฝึก แล้วดูผลจริง

- `fitted_val`: ทุกโมเดลฝึกบน `R_TRAIN_VAL` (ซ่อน 8 คะแนน)
- ตารางหนึ่งแถวต่อคะแนนใน Validation: ผู้ใช้, ภาพยนตร์, ค่าจริง และค่าทำนายทุกโมเดล
- สรุป RMSE / MAE บน Validation เรียงจากดีไปแย่
- ผลจริง: ALS 0.94, GlobalMean 1.40, Item-kNN 1.72 (RMSE)

Part 8: Validation — Top-3 ของทุกโมเดล

```
rng_val = np.random.default_rng(SEED)
VAL_LIKED = VAL[VAL.rating >=
    LIKE_T].groupby("u")["i"].apply(set).to_dict()
rank_view, rank_metric_rows = [], []
for name, m in fitted_val.items():
    precs, recs_, ndcgs, all_top = [], [], [], []
    for u in range(len(USERS)):
        cand = candidates_for(R_TRAIN_VAL, u)
        top = topk_random_tiebreak(m.scores(u), cand, K, rng_val);
            all_top += top
        rel = VAL_LIKED.get(u, set())
        rank_view.append({"user": USERS[u], "model": name,
                          "Top-3": " | ".join(("✓" if j in rel else "") +
                              ITEMS[j] for j in top)})
        if rel:
            precs.append(precision_at_k(top, rel, K));
                recs_.append(recall_at_k(top, rel, K));
                ndcgs.append(ndcg_at_k(top, rel, K))
    rank_metric_rows.append({"Model": name, f"P@{K}": np.mean(precs),
        f"R@{K}": np.mean(recs_), f"nDCG@{K}": np.mean(ndcgs),
                             "Coverage": coverage(all_top)})
print("8.2 Validation — Top-3 ต่อผู้ใช้ (✓ = เรื่องที่ผู้ใช้ชอบใน
    Validation)")
print("Validation liked:", {USERS[u]: [ITEMS[i] for i in s] for u, s in
    VAL_LIKED.items()})
display(pd.DataFrame(rank_view).pivot(index="model", columns="user",
    values="Top-3").loc[MODEL_NAMES])
display(pd.DataFrame(rank_metric_rows).sort_values(f"nDCG@{K}",
    ascending=False).round(4).reset_index(drop=True))
```

อธิบายโค้ด

เทียบรายการแนะนำแบบเห็นภาพ

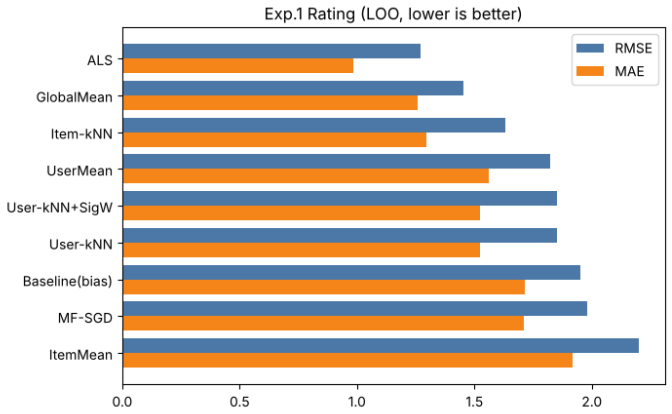
- **VAL_LIKED**: เรื่องที่ผู้ใช้ชอบใน Validation (u1:i1, u2:i6, u3:i11, u8:i6)
- Top-3 ของทุกโมเดล ทุกผู้ใช้ ใส่ ✓ หน้าเรื่องที่ตรงกับ Validation
- **pivot** เป็นตาราง โมเดล × ผู้ใช้ อ่านง่าย
- สรุป P@3, R@3, nDCG@3, Coverage ต่อโมเดล

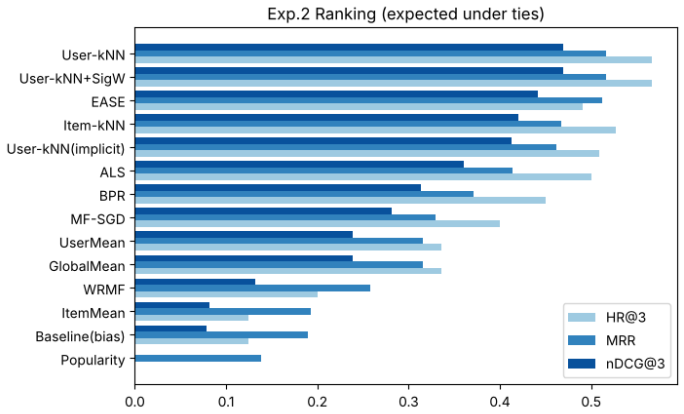
Part 8: ตัวอย่าง Output — Top-3 บน Validation (✓ = ซอบจริง)

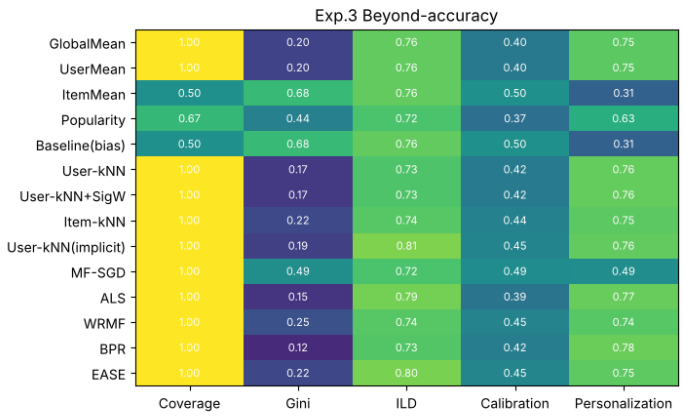
โมเดล	u1	u2	u3	u8
Item-kNN	i9 | ✓i1 | i10	✓i6 | i9 | i12	✓i11 | i5 | i4	✓i6 | i5 | i8
ALS	✓i1 | i8 | i9	✓i6 | i12 | i10	i2 | ✓i11 | i5	i7 | i8 | i9
User-kNN	i9 | i8 | i7	i7 | i8 | i3	✓i11 | i2 | i3	i5 | i9 | i3
EASE	i9 | i5 | i10	i2 | i9 | i11	✓i11 | i6 | i2	i5 | i3 | ✓i6
MF-SGD	i9 | i12 | i11	i9 | i12 | i3	i9 | i3 | ✓i11	i9 | i12 | i5
Popularity	i5 | i12 | i7	i2 | i3 | i12	i5 | i2 | i4	i3 | i12 | i5

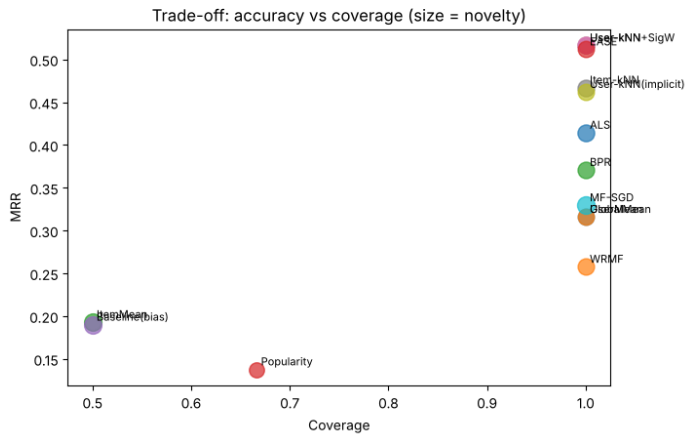
Item-kNN: P@3 0.333, nDCG@3 0.908 บน Validation • MF-SGD ให้ i9 ทุกคน (ค่าเอนเอียงครอบงำ)

Part 9: ตารางสรุปและ Chart









อ่านผล

อ่านกราฟ

- บนซ้าย: RMSE/MAE (ต่ำดี)
- บนขวา: HR/MRR/nDCG แบบค่าคาดหมาย
- ล่างซ้าย: heatmap Beyond-Accuracy
- ล่างขวา: MRR vs Coverage (ขนาดจุด = Novelty)
- โมเดลที่ทำนายคะแนนดี (ALS) ไม่ใช่โมเดลที่จัดอันดับดีที่สุด (User-kNN) — **ต้องเลือกตามงาน**

Part 10: Sensitivity ของไฮเพอร์พารามิเตอร์

```
sens = []
for k_ in [1, 2, 3, 4, 5]:
    for cls in [UserKNN, ItemKNN]:
        s, _ = evaluate_ranking(lambda: cls(k=k_))
        sens.append({"Model": cls.name, "param": f"k={k_}", "MRR":
            s["MRR"], f"HR@{K}": s[f"HR@{K}"]})
for lam in [1, 5, 20, 50]:
    s, _ = evaluate_ranking(lambda: EASE(lam=lam))
    sens.append({"Model": "EASE", "param": f"λ={lam}", "MRR": s["MRR"],
        f"HR@{K}": s[f"HR@{K}"]})
display(pd.DataFrame(sens).pivot(index="param", columns="Model",
    values="MRR").round(4))
# สังเกต: บน MovieMini-12 ค่า k แทบไม่มีผล
เพราะแต่ละรายการมีเพื่อนบ้านเชิงบวกเพียง 1-2ตัว
# และ λ ของ EASE เปลี่ยนขนาดคะแนนแต่ไม่เปลี่ยนลำดับในข้อมูลเล็กนี้ –
บนข้อมูลจริง (LAB 2-3) ค่าเหล่านี้มีผลมาก

mf_rows = []
for reg in [0.02, 0.1, 0.5]:
    m = MF_SGD(k=2, reg=reg, epochs=150).fit(R_FULL)
    train_rmse = rmse([R_FULL[u, i] for u, i in zip(*np.where(known))],
        [m.predict(u, i) for u, i in zip(*np.where(known))])
    res, _ = evaluate_rating(lambda: MF_SGD(k=2, reg=reg, epochs=150))
    mf_rows.append({"λ": reg, "RMSE train": train_rmse, "RMSE LOO":
        res["RMSE"], "MAE LOO": res["MAE"]})
# ตารางที่ 5.12: λ เล็ก → train ดีขึ้นแต่ LOO แย่ลง = overfitting
display(pd.DataFrame(mf_rows).round(4))
```

อธิบายโค้ด

k, λ ของ EASE และ λ ของ MF

- วน k = 1–5 ของ User/Item-kNN และ λ ของ EASE ⟶ MRR แทบไม่เปลี่ยนในข้อมูลเล็กนี้
- MF: λ = 0.02 ⟶ RMSE train 0.04 แต่ LOO 2.45; λ = 0.5 ⟶ LOO 1.98
- เป็นภาพคลาสสิกของ **overfitting** (ตารางที่ 5.12)
- LAB 3 จะปรับจูนบน MovieLens ซึ่งค่าเหล่านี้มีผลจริง

Part 10: เมล็ดสุ่มและ Paired Bootstrap

```
seed_rows = []
for seed in [1, 7, 42, 2569, 2026]:
    for cls in [MF_SGD, WRMF, BPR]:
        s, _ = evaluate_ranking(lambda: cls(seed=seed))
        seed_rows.append({"Model": cls.name, "seed": seed, "MRR":
            s["MRR"], f"HR@{K}": s[f"HR@{K}"]})
sd = pd.DataFrame(seed_rows)
display(sd.groupby("Model")[["MRR", f"HR@{K}"]].agg(["mean",
    "std"]).round(4))

# 10.3 Paired bootstrap ระดับผู้ใช้ (เหมือนหัวข้อ 5.14.2)
def paired_bootstrap(a_name, b_name, B=4000, seed=SEED):
    da = RANK_DETAIL[a_name].groupby("user")["MRR"].mean(); db =
        RANK_DETAIL[b_name].groupby("user")["MRR"].mean()
    diff = (da - db).values; rng = np.random.default_rng(seed)
    boots = [rng.choice(diff, size=len(diff), replace=True).mean() for _
        in range(B)]
    return diff.mean(), np.percentile(boots, [2.5, 97.5])
best = ranking_results.sort_values("MRR",
    ascending=False).Model.tolist()[0]
for other in ["EASE", "Item-kNN", "MF-SGD", "Popularity"]:
    if other == best: continue
    m_, ci = paired_bootstrap(best, other)
    verdict = "→ CI คร่อมศูนย์: ยังสรุปไม่ได้ว่าต่างกันจริง" if ci[0] <=
        0 <= ci[1] else "→ CI ไม่คร่อมศูนย์: ต่างอย่างมีหลักฐาน"
    print(f"{best} - {other}: mean ΔRR = {m_:.4f}, 95% CI = [{ci[0]:.4f},
        {ci[1]:.4f}] {verdict}")
# สังเกต: ผู้ใช้มีเพียง 8 คนช่วงความเชื่อมั่นจึงกว้าง –
อย่าสรุปผู้ชนะจากค่าเฉลี่ยอย่างเดียว
```

อธิบายโค้ด

ความต่างเชื่อได้แค่ไหน

- MF / WRMF / BPR รัน 5 seed ⟶ BPR MRR 0.42 ± 0.06 (แกว่งมาก)
- Bootstrap ระดับผู้ใช้ 4,000 รอบ บนค่าเฉลี่ย RR รายผู้ใช้
- User-kNN – EASE: CI [−0.04, 0.03] ⟶ **ยังสรุปไม่ได้**
- User-kNN – MF-SGD: CI [0.06, 0.31] ⟶ ต่างอย่างมีหลักฐาน

Part 11: GUI — ข้อมูลผู้ใช้

```
def user_info_md(u, R):
    hist = np.where(~np.isnan(R[u]))[0]
    liked = [j for j in hist if R[u, j] >= LIKE_T]
    gcount = pd.Series([g for j in liked for g in
        GENRE[ITEMS[j]].split("|")]).value_counts()
    sims = sorted([(pearson_users(R, u, v)[0],
        USERS[v]) for v in range(len(USERS)) if v !=
        u], reverse=True)[:3]
    return (f"### ผู้ใช้ {USERS[u]}\n"
            f"- จำนวนคะแนน: **{len(hist)}** ·
ค่าเฉลี่ย: **{np.nanmean(R[u]):.2f}** ·
ชอบ (≥4): **{len(liked)}**เรื่อง\n"
            f"- แนวที่ชอบ: {', '.join(f'{g} ({c})' for
                g, c in gcount.items()) or '-'}\n"
            f"- เพื่อนบ้านที่คล้ายที่สุด (Pearson): {',
                '.join(f'{n} ({s:.2f})' for s, n in
                sims)}")
```

อธิบายโค้ด

สรุปผู้ใช้เป็น Markdown

- จำนวนคะแนน ค่าเฉลี่ย และจำนวนเรื่องที่ชอบ
- นับแนวภาพยนตร์จากเรื่องที่ชอบ ⟶ แนวที่ชอบมากที่สุด
- เพื่อนบ้าน Pearson 3 คน ⟶ อธิบายว่า User-kNN จะยืมคะแนนจากใคร

Part 11: GUI — สร้างผลแนะนำและตารางเทียบ

```
def gui_recommend(user_id, model_name, k, mode):
    u = U_INDEX[user_id]; k = int(k)
    R = R_TRAIN_VAL if mode == "Validation (ฝึกบนชุดฝึก)" else R_FULL
    hidden = VAL[(VAL.u == u)] if R is R_TRAIN_VAL else VAL.iloc[0:0]
    hidden_like = set(hidden[hidden.rating >= LIKE_T].i)
    hist = pd.DataFrame([{"Movie": f"{ITEMS[j]} {TITLE[ITEMS[j]]}",
        "Genre": GENRE[ITEMS[j]], "Rating": R[u, j]}
                         for j in np.where(~np.isnan(R[u]))[0]]).sort_values
                             ("Rating", ascending=False)
    models = {m.name: m.fit(R) for m in make_models()}
    m = models[model_name]; cand = candidates_for(R, u); sc = m.scores(u)
    top = topk_random_tiebreak(sc, cand, k, np.random.default_rng(SEED))
    rec = pd.DataFrame([{"Rank": n, "Movie": f"{ITEMS[j]}
        {TITLE[ITEMS[j]]}", "Genre": GENRE[ITEMS[j]],
                         "Score": round(float(sc[j]), 4),
                         "Pred. rating": round(m.predict(u, j), 2) if
                             m.rating_model else "-",
                         "Why": m.explain(u, j) if hasattr(m, "neighbors")
                             else "",
                         "Hidden & liked": "✓" if j in hidden_like else ""}
                             for n, j in enumerate(top, 1)])
    comp = pd.DataFrame({name: [("✓" if j in hidden_like else "") +
        f"{ITEMS[j]} {TITLE[ITEMS[j]]}"
                                for j in topk_random_tiebreak(mm.scores(u),
                                    cand, k, np.random.default_rng(SEED))]
                         for name, mm in models.items()}, index=[f"#{n}"
                             for n in range(1,
                             k+1)]).T.reset_index(names="Model")
    hid = (", ".join(f"{ITEMS[i]} {TITLE[ITEMS[i]]} (r={r:.0f})" for i, r
        in zip(hidden.i, hidden.rating))) or "-"
    return user_info_md(u, R) + f"\n- Validation ที่ซ่อนไว้: {hid}", hist,
        rec, comp
```

อธิบายโค้ด

หนึ่งฟังก์ชัน สี่ผลลัพธ์

- `user_info_md`: จำนวนคะแนน ค่าเฉลี่ย แนวที่ชอบ และเพื่อนบ้าน Pearson 3 คน
- โหมด **Validation**: ฝึกบนชุดฝึก แล้วติ๊ก ✓ เมื่อแนะนำเรื่องที่ซ่อนและผู้ใช้ชอบ
- ตารางแนะนำมี Score, ค่าทำนาย และ **Why** (เพื่อนบ้าน/รายการที่ใช้)
- `comp`: Top-K ของทุกโมเดลสำหรับผู้ใช้คนเดียวกัน

Part 11: GUI — จัดหน้าด้วย gr.Blocks

```
with gr.Blocks(title="Chapter 5 – Collaborative Filtering
    (MovieMini-12)") as demo:
    gr.Markdown("## Chapter 5 – Collaborative Filtering
        Recommender (MovieMini-12)")
    with gr.Row():
        dd_user = gr.Dropdown(USERS, value=USERS[0],
            label="User")
        dd_model = gr.Dropdown(MODEL_NAMES, value="User-kNN",
            label="Model")
        sl_k = gr.Slider(1, 6, 3, step=1, label="Top-K")
        rd_mode = gr.Radio(["Full data", "Validation
            (ฝึกบนชุดฝึก)"], value="Full data", label="Mode")
    btn = gr.Button("Recommend", variant="primary")
    md_user = gr.Markdown()
    with gr.Row():
        df_hist = gr.Dataframe(label="User History")
        df_rec = gr.Dataframe(label="Recommended Movies")
    df_comp = gr.Dataframe(label="Top-K ของทุกโมเดล
        (เทียบกัน)")
    btn.click(gui_recommend, [dd_user, dd_model, sl_k,
        rd_mode], [md_user, df_hist, df_rec, df_comp])
    demo.load(gui_recommend, [dd_user, dd_model, sl_k,
        rd_mode], [md_user, df_hist, df_rec, df_comp])

demo.launch(debug=False)
```

อธิบายโค้ด

Layout ของแอป

- แถวบน: เลือก User, Model, Top-K และโหมด
- ปุ่ม Recommend ⟶ ข้อมูลผู้ใช้ + History + Recommended + ตารางเทียบ
- **demo.load** แสดงผลทันทีเมื่อเปิดแอป
- แนวทางใช้: เลือก ul ⟶ สลับ User-kNN / Item-kNN / EASE ⟶ ดูว่าเหตุผลต่างกันอย่างไร

สรุปแนวคิดของ Source Code LAB 1

Take-away

- การกรองร่วมใช้ **เมทริกซ์คะแนนอย่างเดียว** แต่เชื่อมข้ามแนวได้ (u5: The Wild Robot ⟶ Conclave)
- ทุกโมเดลอยู่ใต้ interface `fit / predict / scores` เดียวกัน ⟶ เพิ่มโมเดลหรือเปลี่ยน protocol ได้โดยไม่แก้ส่วนอื่น
- Protocol Card + ฝึกใหม่ทุก fold + ค่าคาดหมายเมื่อคะแนนเสมอ ทำให้ผลตรงตารางที่ 5.16–5.17
- บนข้อมูลเล็ก วิธีง่ายชนะ: GlobalMean ด้าน rating, User-kNN/EASE ด้าน ranking — และ CI ยังกว้าง
- Beyond-Accuracy + ตัวอย่าง Output บน Validation ช่วยให้เห็นสิ่งที่ค่าเฉลี่ยซ่อนไว้

LAB 1 = พื้นฐานบน MovieMini-12 • LAB 2 = MovieLens + Surprise หลายโปรโตคอล • LAB 3 = Pipeline + ปรับจูน